

Informações – Método Deep Learning

O trabalho prático tem como objetivo realizar um estudo de um órgão/estrutura específico através da segmentação de imagens médicas. O tema centra-se na segmentação da porção intra-hepática da veia umbilical em imagens de ultrassom fetal usando um método de *Deep Learning*, nomeadamente a *UNet*.



Para garantir que todos os alunos dispõem de recursos computacionais adequados para treinar e testar o método de Deep Learning, será utilizado o Google Colab para a componente de segmentação. As etapas de pré-processamento e pós-processamento deverão ser implementadas em Jupyter Notebook, à semelhança do que tem sido realizado nas worksheets.

1. Descrição do Google Colab

O Google Colab é uma plataforma online gratuita que permite executar código em *Python* diretamente no navegador, sem necessidade de instalação local. Baseia-se em ambientes de notebooks semelhantes ao Jupyter Notebook e disponibiliza acesso a recursos computacionais na cloud, incluindo GPUs e TPUs, o que o torna especialmente adequado para tarefas de Deep Learning.

Nesta unidade curricular, o Colab será utilizado para treinar e testar os modelos de segmentação.

2. Instruções de utilização do código fornecido no Google Colab

1. Abrir a página do Google Colab <https://colab.research.google.com/> e iniciar sessão com uma conta Gmail;
2. No Google Drive da mesma conta Gmail, adicionar a pasta *Dataset* fornecida (contendo também as pastas dos pré-processamentos aplicados);
3. Importar o ficheiro *FetalVeinSegmentationUS.ipynb* para o Google Colab. A execução do código é similar à do Jupyter Notebook;
4. Executar o código, com as devidas alterações a desenvolver durante o trabalho prático.

3. Informação adicional

- Permitir o acesso do ficheiro Colab à Drive onde está alocado o dataset

```
#import data from google drive
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

- Diretoria do dataset após o acesso

```
data_dir = "/content/drive/MyDrive/Dataset"
```

- Seleção do dataset a processar (original, pré-processamento 1, pré-processamento 2, ...)

O dataset onde o modelo será treinado e testado é definido na linha

```
data_dir = "/content/drive/MyDrive/Dataset"  
images = sorted(glob.glob(os.path.join(data_dir, "images", "*.png")))
```

Neste caso, o dataset seria as imagens originais. Para alterar para outro dataset, por exemplo para o teste com um dataset com brilho alterado, a path "images" deveria ser alterada para "images_brilho"

- Guardar os modelos de Deep Learning para cada teste

Os modelos serão armazenados em ficheiros *.pth numa pasta "drive/MyDrive/Save_Models" com o nome exemplo "best_metric_model.pth" e lidos posteriormente para a fase de teste

Gravar:

```
torch.save(model.state_dict(), os.path.join('drive/MyDrive/Save_Models', "best_metric_model.pth"))
```

Ler:

```
model.load_state_dict(torch.load(os.path.join('drive/MyDrive/Save_Models', "best_metric_model.pth")))
```

- Os alunos deverão criar a pasta "Save_Models" na Drive

- De forma a guardar os modelos *.pth para cada experiência, é aconselhável utilizar nomes diferentes para cada teste (exemplo para o teste com um dataset com brilho alterado: "best_metric_model_brilho.pth"), alterando nas linhas de gravar e ler anteriores.

- Guardar das máscaras de segmentação

Após o treino, as máscaras que resultam da previsão da rede serão armazenadas na pasta "/content/drive/MyDrive/Dataset/results", sendo que os alunos deverão criar essa pasta dentro da pasta datasets.

Para alterar para outro dataset, por exemplo para o teste com um dataset com brilho alterado, terá que ser criada uma pasta de resultados para esse teste "results_brilho" com a alteração da linha de código

```
name_img = name_img['filename_or_obj'][0].replace("images", "results")  
para  
name_img = name_img['filename_or_obj'][0].replace("images", "results_brilho")
```